Lab 2: Convolutional Neural Networks for Computer Vision

In this lab, we will learn how to use CNNs for computer vision applications. We will look at two applications - 1) detecting objects in images, and 2) detecting cars in images.

Grading Breakdown:

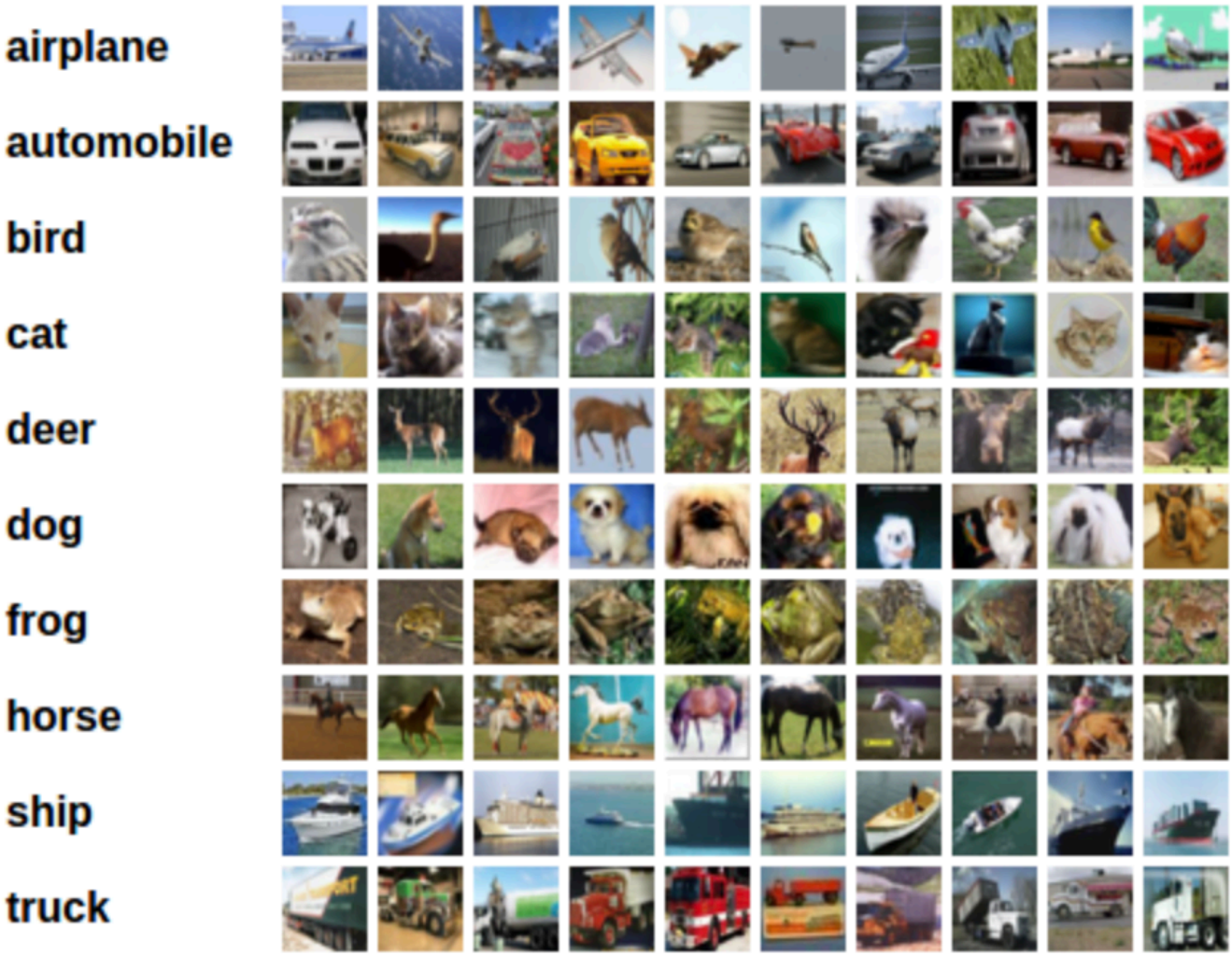
Part 1: Detecting objects 40%

Part 2: Detecting cars with new model 30%

Part 3: Detecting cars with fine-tuned model 30%

Part 1: CNN to Detect Images of 10 Objects

In this section, we will design a CNN to classify images of 10 objects from the CIFAR10 dataset.



In [2]:
```
# Import packages first
import torch
import torchvision
import torchvision.transforms as transforms
```

In [3]:
```
# Just like in Lab 1, we willnormalize our inputs. As we go along, notice the recurring themes in deep learning, such
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

# set a batch size
```

```
batch_size = 4

# download the train set
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
# we'll just split the train into train and val
trainset, valset = torch.utils.data.random_split(trainset, [0.8, 0.2])
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True, num_workers=2)
valloader = torch.utils.data.DataLoader(valset, batch_size=batch_size, shuffle=True, num_workers=2)

# download the train set
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(
    testset, batch_size=batch_size, shuffle=False, num_workers=0)

# labels. In this problem there are 10 labels.
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

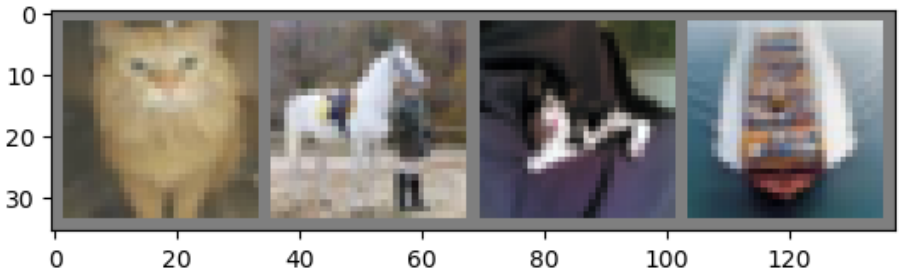
In [5]:

```
# Visualize a few images in the train set
import matplotlib.pyplot as plt
import numpy as np

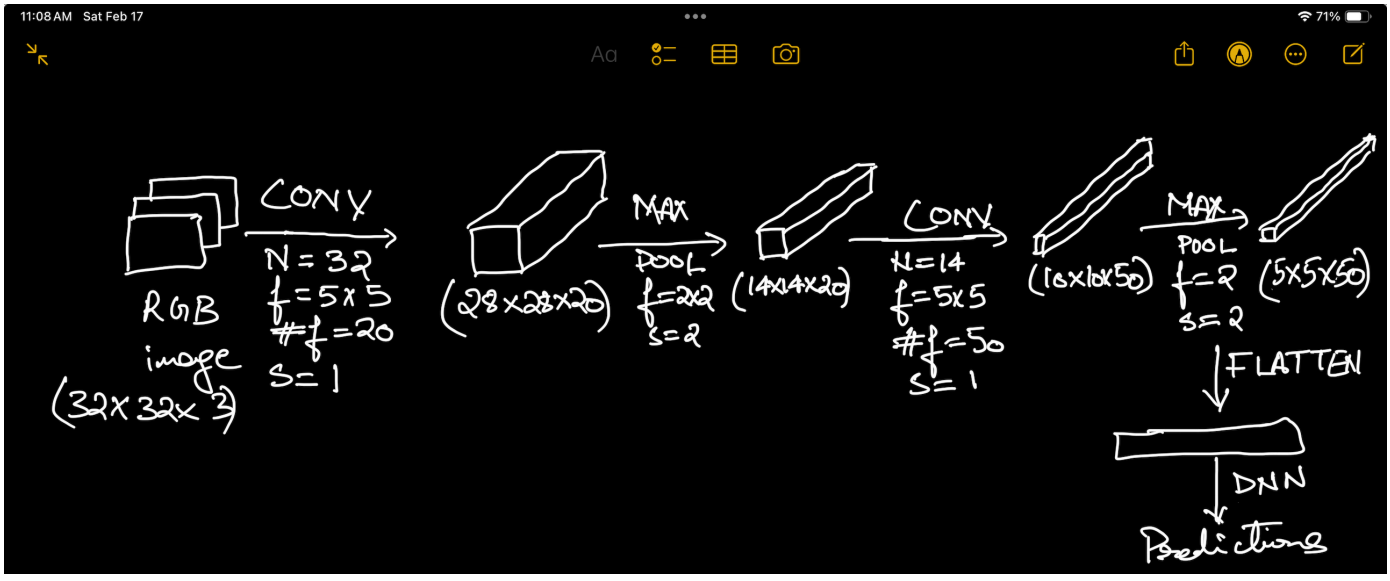
def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# get some random training images
trainloader.num_workers = 0
dataiter = iter(trainloader)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```




```
cat   horse cat   ship
```



In [6]:

```
# CNN Architecture
import torch.nn as nn
```

```
import torch.nn.functional as F

class MyObjDetectorCNN(nn.Module):
    def __init__(self):
        super(MyObjDetectorCNN, self).__init__()

        # this is the conv net part
        self.convolutional_layer = nn.Sequential(
            # we can use the equation ((N + 2p - f) / s) + 1 to quickly compute the output of a convolution op.
            # however, PyTorch makes it easy for us, since it computes this equation for us. All we have to do is tel
            # in the input and the output. For the first convolution layer, the in_channels would be the channels a s
            # number of channels in the output to be 20. We want to use 5X5 convolutions and single strides. Write th
            nn.Conv2d(in_channels = 3, out_channels = 20, kernel_size = 5, stride = 1),
            nn.ReLU(),
            # Lets subsample after convolution. We will use Max Pooling in this example.
            nn.MaxPool2d(kernel_size = 2,stride = 2),
            # Let's add another convolutional layer. We want the output of this convolution operation to have 50 char
            nn.Conv2d(in_channels = 20, out_channels = 50, kernel_size = 5, stride = 1),
            nn.ReLU(),
            # Subsample again
            nn.MaxPool2d(kernel_size = 2, stride = 2),

        )

        # this is the classifier head. So the model is actually CNN + DNN. We will flatten the output from the CNN ab
        self.linear_layer = nn.Sequential(
            nn.Linear(in_features = 1250, out_features = 500),
            nn.ReLU(),
            nn.Linear(in_features = 500, out_features = 10), # observe how we have output features = 10. There are 10
        )


    def forward(self, x): # x = image 32x32x3

        x = self.convolutional_layer(x)
        x = torch.flatten(x, 1) # Flattening the output of the CNN for the DNN
        x = self.linear_layer(x)
        return x

net = MyObjDetectorCNN()
```

In [7]:
```
# define a loss function and opimizer
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr = 0.001, momentum = 0.9)
```

In [8]:
```
# train the network (better use a GPU for this, look at the first lab for moving objects to GPU)
net.train()
for epoch in range(2):  # loop over the dataset multiple times, you could increase this.

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
            running_loss = 0.0

print('Finished Training')
```

```
[1,  2000] loss: 1.942
[1,  4000] loss: 1.586
[1,  6000] loss: 1.467
[1,  8000] loss: 1.357
[1, 10000] loss: 1.273
[2,  2000] loss: 1.175
[2,  4000] loss: 1.156
[2,  6000] loss: 1.100
[2,  8000] loss: 1.082
[2, 10000] loss: 1.049
Finished Training
```

In [9]:

```python
# test your CNN on the test set
# Note: If your accuracy is low, you need to further train your CNN.
dataiter = iter(testloader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in range(4)))

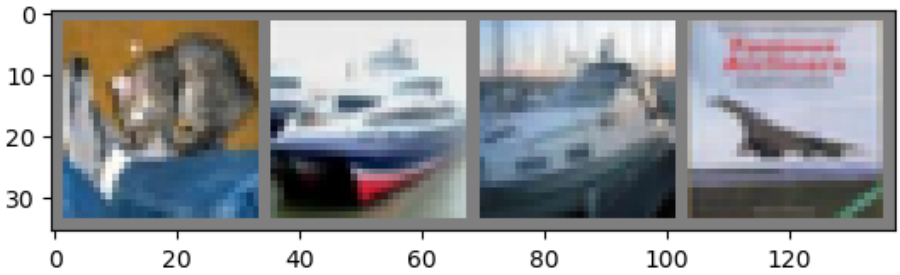
# What is the accuracy, precision, recall and F1-score on the test dataset?
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

y_test = []
y_test_predictions = []

net.eval()
with torch.no_grad():
    for i, data in enumerate(testloader, 0):
        inputs, labels = data
        y_test.extend(labels.tolist())

        outputs = net(inputs)
        y_test_predictions.extend(torch.argmax(outputs, dim=1).tolist())

# print accuracy, prec, rec, f1-score here
print("Accuracy:", accuracy_score(y_test, y_test_predictions))
print("Precision:", precision_score(y_test, y_test_predictions, average='macro', zero_division=0))
print("Recall:", recall_score(y_test, y_test_predictions, average='macro', zero_division=0))
print("F1-score:", f1_score(y_test, y_test_predictions, average='macro', zero_division=0))
```




```
GroundTruth:  cat   ship  ship  plane
Accuracy: 0.6369
Precision: 0.6424346834000492
Recall: 0.6369
F1-score: 0.633397404546739
```

Part 2: Using a new CNN to Detect Cars in Images

With previous lab learning, you should have some knowledge about how to develop a CNN to detect images of different types. In this lab, we will learn how a CNN can be fine tuned to detect car images. We will use a publicly available test dataset of car images, and train a detection model with two different modalities:

1. Train a new model for this dataset.

2. Fine-tune the model you built in Part 1 on this dataset.
3. Compare the performance of the two models.

First, we need to download the cars dataset.

In [11]:
```
 # download the dataset
import kagglehub

# Download latest version
path = kagglehub.dataset_download("cyizhuo/stanford-cars-by-classes-folder")

print("Path to dataset files:", path)
```

```
Path to dataset files: C:\Users\justi\.cache\kagglehub\datasets\cyizhuo\stanford-cars-by-classes-folder\versions\5
```

In [12]:
```
# Code to load the dataset:
from pathlib import Path
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder
import torchvision.transforms as T

root = Path(path)  # contains train/ and test/

tf_train = T.Compose([
    T.RandomResizedCrop(224, scale=(0.7, 1.0)),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406],
                [0.229, 0.224, 0.225]),
])

tf_test = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406],
                [0.229, 0.224, 0.225]),
])

train_ds = ImageFolder(root / "train", transform=tf_train)
test_ds  = ImageFolder(root / "test",  transform=tf_test)

train_loader = DataLoader(train_ds, batch_size=4, shuffle=True,  num_workers=0)
test_loader  = DataLoader(test_ds,  batch_size=4, shuffle=False, num_workers=0)

# Class names and mapping
print(train_ds.classes[:5])        # e.g., ['Acura', 'Aston Martin', ...]
print(train_ds.class_to_idx)       # {'Acura': 0, 'Aston Martin': 1, ...}
```

```
['AM General Hummer SUV 2000', 'Acura Integra Type R 2001', 'Acura RL Sedan 2012', 'Acura TL Sedan 2012', 'Acura TL Type-S 2008']
{'AM General Hummer SUV 2000': 0, 'Acura Integra Type R 2001': 1, 'Acura RL Sedan 2012': 2, 'Acura TL Sedan 2012': 3, 'Acura TL Type-S 2008': 4, 'Acura TSX Sedan 2012': 5, 'Acura ZDX Hatchback 2012': 6, 'Aston Martin V8 Vantage Convertible 2012': 7, 'Aston Martin V8 Vantage Coupe 2012': 8, 'Aston Martin Virage Convertible 2012': 9, 'Aston Martin Virage Coupe 2012': 10, 'Audi 100 Sedan 1994': 11, 'Audi 100 Wagon 1994': 12, 'Audi A5 Coupe 2012': 13, 'Audi R8 Coupe 2012': 14, 'Audi RS 4 Convertible 2008': 15, 'Audi S4 Sedan 2007': 16, 'Audi S4 Sedan 2012': 17, 'Audi S5 Convertible 2012': 18, 'Audi S5 Coupe 2012': 19, 'Audi S6 Sedan 2011': 20, 'Audi TT Hatchback 2011': 21, 'Audi TT RS Coupe 2012': 22, 'Audi TTS Coupe 2012': 23, 'Audi V8 Sedan 1994': 24, 'BMW 1 Series Convertible 2012': 25, 'BMW 1 Series Coupe 2012': 26, 'BMW 3 Series Sedan 2012': 27, 'BMW 3 Series Wagon 2012': 28, 'BMW 6 Series Convertible 2007': 29, 'BMW ActiveHybrid 5 Sedan 2012': 30, 'BMW M3 Coupe 2012': 31, 'BMW M5 Sedan 2010': 32, 'BMW M6 Convertible 2010': 33, 'BMW X3 SUV 2012': 34, 'BMW X5 SUV 2007': 35, 'BMW X6 SUV 2012': 36, 'BMW Z4 Convertible 2012': 37, 'Bentley Arnage Sedan 2009': 38, 'Bentley Continental Flying Spur Sedan 2007': 39, 'Bentley Continental GT Coupe 2007': 40, 'Bentley Continental GT Coupe 2012': 41, 'Bentley Continental Supersports Conv. Convertible 2012': 42, 'Bentley Mulsanne Sedan 2011': 43, 'Bugatti Veyron 16.4 Convertible 2009': 44, 'Bugatti Veyron 16.4 Coupe 2009': 45, 'Buick Enclave SUV 2012': 46, 'Buick Rainier SUV 2007': 47, 'Buick Regal GS 2012': 48, 'Buick Verano Sedan 2012': 49, 'Cadillac CTS-V Sedan 2012': 50, 'Cadillac Escalade EXT Crew Cab 2007': 51, 'Cadillac SRX SUV 2012': 52, 'Chevrolet Avalanche Crew Cab 2012': 53, 'Chevrolet Camaro Convertible 2012': 54, 'Chevrolet Cobalt SS 2010': 55, 'Chevrolet Corvette Convertible 2012': 56, 'Chevrolet Corvette Ron Fellows Edition Z06 2007': 57, 'Chevrolet Corvette ZR1 2012': 58, 'Chevrolet Express Cargo Van 2007': 59, 'Chevrolet Express Van 2007': 60, 'Chevrolet HHR SS 2010': 61, 'Chevrolet Impala Sedan 2007': 62, 'Chevrolet Malibu Hybrid Sedan 2010': 63, 'Chevrolet Malibu Sedan 2007': 64, 'Chevrolet Monte Carlo Coupe 2007': 65, 'Chevrolet Silverado 1500 Classic Extended Cab 2007': 66, 'Chevrolet Silverado 1500 Extended Cab 2012': 67, 'Chevrolet Silverado 1500 Hybrid Crew Cab 2012': 68, 'Chevrolet Silverado 1500 Regular Cab 2012': 69, 'Chevrolet Silverado 2500HD Regular Cab 2012': 70, 'Chevrolet Sonic Sedan 2012': 71, 'Chevrolet Tahoe Hybrid SUV 2012': 72, 'Chevrolet TrailBlazer SS 2009': 73, 'Chevrolet Traverse SUV 2012': 74, 'Chrysler 300 SRT-8 2010': 75, 'Chrysler Aspen SUV 2009': 76, 'Chrysler Crossfire Convertible 2008': 77, 'Chrysler PT Cruiser Convertible 2008': 78, 'Chrysler Sebring Convertible 2010': 79, 'Chrysler Town and Country Minivan 2012': 80, 'Daewoo Nubira Wagon 2002': 81, 'Dodge Caliber Wagon 2007': 82, 'Dodge Caliber Wagon 2012': 83, 'Dodge Caravan Minivan 1997': 84, 'Dodge Challenger SRT8 2011': 85, 'Dodge Charger SRT-8 2009': 86, 'Dodge Charger Sedan 2012': 87, 'Dodge Dakota Club Cab 2007': 88, 'Dodge Dakota Crew Cab 2010': 89, 'Dodge Durango SUV 2007': 90, 'Dodge Durango SUV 2012': 91, 'Dodge Journey SUV 2012': 92, 'Dodge Magnum Wagon 2008': 93, 'Dodge Ram Pickup 3500 Crew Cab 2010': 94, 'Dodge Ram Pickup 3500 Quad Cab 2009': 95, 'Dodge Sprinter Cargo Van 2009': 96, 'Eagle Talon Hatchback 1998': 97, 'FIAT 500 Abarth 2012': 98, 'FIAT 500 Convertible 2012': 99, 'Ferrari 458 Italia Convertible 2012': 100, 'Ferrari 458 Italia Coupe 2012': 101, 'Ferrari California Convertible 2012': 102, 'Ferrari FF Coupe 2012': 103, 'Fisker Karma Sedan 2012': 104, 'Ford E-Series Wagon Van 2012': 105, 'Ford Edge SUV 2012': 106, 'Ford Expedition EL SUV 2009': 107, 'Ford F-150 Regular Cab 2007': 108, 'Ford F-150 Regular Cab 2012': 109, 'Ford F-450 Super Duty Crew Cab 2012': 110, 'Ford Fiesta Sedan 2012': 111, 'Ford Focus Sedan 2007': 112, 'Ford Freestar Minivan 2007': 113, 'Ford GT Coupe 2006': 114, 'Ford Mustang Convertible 2007': 115, 'Ford Ranger SuperCab 2011': 116, 'GMC Acadia SUV 2012': 117, 'GMC Canyon Extended Cab 2012': 118, 'GMC Savana Van 2012': 119, 'GMC Terrain SUV 2012': 120, 'GMC Yukon Hybrid SUV 2012': 121, 'Geo Metro Convertible 1993': 122, 'HUMMER H2 SUT Crew Cab 2009': 123, 'HUMMER H3T Crew Cab 2010': 124, 'Honda Accord Coupe 2012': 125, 'Honda Accord Sedan 2012': 126, 'Honda Odyssey Minivan 2007': 127, 'Honda Odyssey Minivan 2012': 128, 'Hyundai Accent Sedan 2012': 129, 'Hyundai Azera Sedan 2012': 130, 'Hyundai Elantra Sedan 2007': 131, 'Hyundai Elantra Touring Hatchback 2012': 132, 'Hyundai Genesis Sedan 2012': 133, 'Hyundai Santa Fe SUV 2012': 134, 'Hyundai Sonata Hybrid Sedan 2012': 135, 'Hyundai Sonata Sedan 2012': 136, 'Hyundai Tucson SUV 2012': 137, 'Hyundai Veloster Hatchback 2012': 138, 'Hyundai Veracruz SUV 2012': 139, 'Infiniti G Coupe IPL 2012': 140, 'Infiniti QX56 SUV 2011': 141, 'Isuzu Ascender SUV 2008': 142, 'Jaguar XK XKR 2012': 143, 'Jeep Compass SUV 2012': 144, 'Jeep Grand Cherokee SUV 2012': 145, 'Jeep Liberty SUV 2012': 146, 'Jeep Patriot SUV 2012': 147, 'Jeep Wrangler SUV 2012': 148, 'Lamborghini Aventador Coupe 2012': 149, 'Lamborghini Diablo Coupe 2001': 150, 'Lamborghini Gallardo LP 570-4 Superleggera 2012': 151, 'Lamborghini Reventon Coupe 2008': 152, 'Land Rover LR2 SUV 2012': 153, 'Land Rover Range Rover SUV 2012': 154, 'Lincoln Town Car Sedan 2011': 155, 'MINI Cooper Roadster Convertible 2012': 156, 'Maybach Landaulet Convertible 2012': 157, 'Mazda Tribute SUV 2011': 158, 'McLaren MP4-12C Coupe 2012': 159, 'Mercedes-Benz 300-Class Convertible 1993': 160, 'Mercedes-Benz C-Class Sedan 2012': 161, 'Mercedes-Benz E-Class Sedan 2012': 162, 'Mercedes-Benz S-Class Sedan 2012': 163, 'Mercedes-Benz SL-Class Coupe 2009': 164, 'Mercedes-Benz Sprinter Van 2012': 165, 'Mitsubishi Lancer Sedan 2012': 166, 'Nissan 240SX Coupe 1998': 167, 'Nissan Juke Hatchback 2012': 168, 'Nissan Leaf Hatchback 2012': 169, 'Nissan NV Passenger Van 2012': 170, 'Plymouth Neon Coupe 1999': 171, 'Porsche Panamera Sedan 2012': 172, 'Ram C_V Cargo Van Minivan 2012': 173, 'Rolls-Royce Ghost Sedan 2012': 174, 'Rolls-Royce Phantom Drophead Coupe Convertible 2012': 175, 'Rolls-Royce Phantom Sedan 2012': 176, 'Scion xD Hatchback 2012': 177, 'Spyker C8 Convertible 2009': 178, 'Spyker C8 Coupe 2009': 179, 'Suzuki Aerio Sedan 2007': 180, 'Suzuki Kizashi Sedan 2012': 181, 'Suzuki SX4 Hatchback 2012': 182, 'Suzuki SX4 Sedan 2012': 183, 'Tesla Model S Sedan 2012': 184, 'Toyota 4Runner SUV 2012': 185, 'Toyota Camry Sedan 2012': 186, 'Toyota Corolla Sedan 2012': 187, 'Toyota Sequoia SUV 2012': 188, 'Volkswagen Beetle Hatchback 2012': 189, 'Volkswagen Golf Hatchback 1991': 190, 'Volkswagen Golf Hatchback 2012': 191, 'Volvo 240 Sedan 1993': 192, 'Volvo C30 Hatchback 2012': 193, 'Volvo XC90 SUV 2007': 194, 'smart fortwo Convertible 2012': 195}
```

Construct a new model to train on cars

In [35]:
```python
# You could use your CNN from Part 1, or a different architecture.
# CNN Architecture
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models
```

```
# Load pretrained VGG16
orig = models.vgg16(weights="IMAGENET1K_V1")

# Freeze the convolutional layers
for p in orig.features.parameters():
    p.requires_grad = False

# Replace the final layer for the 196 car classes
in_features = orig.classifier[6].in_features
orig.classifier[6] = nn.Linear(in_features, len(train_ds.classes))

# Loss function
criterion = nn.CrossEntropyLoss()

# IMPORTANT: Create the optimizer AFTER replacing the final layer
optimizer = optim.SGD(
    filter(lambda p: p.requires_grad, orig.parameters()),
    lr=0.001,
    momentum=0.9
)

print("Number of output classes:", orig.classifier[6].out_features)
print("Optimizer connected to final layer:",
      all(any(p is q for group in optimizer.param_groups for q in group["params"])
          for p in orig.classifier[6].parameters()))
```

```
Number of output classes: 196
Optimizer connected to final layer: True
```

In [36]:

```
# Train the network
orig.train()

for epoch in range(2):
    running_loss = 0.0

    for i, data in enumerate(train_loader, 0):
        inputs, labels = data

        optimizer.zero_grad()

        outputs = orig(inputs)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        if i % 200 == 199:
            print(
                f'[{epoch + 1}, {i + 1:5d}] '
                f'loss: {running_loss / 200:.3f}'
            )
            running_loss = 0.0

print('Finished Training')
```

```
[1,   200] loss: 5.180
[1,   400] loss: 4.899
[1,   600] loss: 4.827
[1,   800] loss: 4.553
[1,  1000] loss: 4.502
[1,  1200] loss: 4.443
[1,  1400] loss: 4.460
[1,  1600] loss: 4.345
[1,  1800] loss: 4.342
[1,  2000] loss: 4.244
[2,   200] loss: 4.048
[2,   400] loss: 3.976
[2,   600] loss: 4.066
[2,   800] loss: 4.060
[2,  1000] loss: 4.134
[2,  1200] loss: 4.180
[2,  1400] loss: 4.135
[2,  1600] loss: 4.172
[2,  1800] loss: 4.113
[2,  2000] loss: 4.142
Finished Training
```

In [37]:

```python
# Calculate accuracy, precision, recall and F1 score of the model

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

orig.eval()

all_labels = []
all_predictions = []

with torch.no_grad():
    for i, (inputs, labels) in enumerate(test_loader):
        outputs = orig(inputs)
        _, predicted = torch.max(outputs, 1)

        all_labels.extend(labels.numpy())
        all_predictions.extend(predicted.numpy())

        if i >= 499:
            break

accuracy = accuracy_score(all_labels, all_predictions)
precision = precision_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)
recall = recall_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)
f1 = f1_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)

print(f"Test images evaluated: {len(all_labels)}")
print(f"Test Accuracy: {accuracy:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall: {recall:.2%}")
print(f"F1 Score: {f1:.2%}")
```

```
Test images evaluated: 2000
Test Accuracy: 10.60%
Precision: 24.68%
Recall: 10.60%
F1 Score: 11.48%
```

Part 3: Fine-tune a model

In [18]:

```python
from torchvision import models
orig = models.vgg16(pretrained = True)
```

```
C:\Users\justi\anaconda3\Lib\site-packages\torchvision\models\_utils.py:207: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
C:\Users\justi\anaconda3\Lib\site-packages\torchvision\models\_utils.py:222: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=VGG16_Weights.IMAGENET1K_V1`. You can also use `weights=VGG16_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
```

In [19]:
```
# Peek into vgg16 model
orig
```

Out[19]:
```
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU(inplace=True)
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): ReLU(inplace=True)
    (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (8): ReLU(inplace=True)
    (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): ReLU(inplace=True)
    (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (15): ReLU(inplace=True)
    (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (18): ReLU(inplace=True)
    (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (20): ReLU(inplace=True)
    (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (24): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (25): ReLU(inplace=True)
    (26): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (27): ReLU(inplace=True)
    (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (29): ReLU(inplace=True)
    (30): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=4096, bias=True)
    (4): ReLU(inplace=True)
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

In [21]:
```
# Freeze the conv layers
for p in orig.features.parameters(): # what is the conv layer called? Look at the architecture above
    p.requires_grad = False
```

In [22]:
```
# Change the number of neurons in the last linear layer to what the number of cars
in_features = orig.classifier[6].in_features
orig.classifier[6] = nn.Linear(in_features, 196) # what is the index of the last linear layer? and what should it be
```

In [38]:
```
# Continue training the VGG16 network

orig.train()

for epoch in range(1):
    running_loss = 0.0
```

```
    batches = 0

    for i, data in enumerate(train_loader, 0):
        inputs, labels = data

        optimizer.zero_grad()

        outputs = orig(inputs)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        batches += 1

        if i % 200 == 199:
            print(
                f'[Continue, {i + 1:5d}] '
                f'loss: {running_loss / 200:.3f}'
            )
            running_loss = 0.0
            batches = 0

    if batches > 0:
        print(f'Remaining batches loss: {running_loss / batches:.3f}')

print('Finished Training')
```

```
[Continue,   200] loss: 3.849
[Continue,   400] loss: 3.923
[Continue,   600] loss: 4.022
[Continue,   800] loss: 3.881
[Continue,  1000] loss: 4.014
[Continue,  1200] loss: 3.973
[Continue,  1400] loss: 4.004
[Continue,  1600] loss: 4.135
[Continue,  1800] loss: 4.082
[Continue,  2000] loss: 4.123
Remaining batches loss: 3.917
Finished Training
```

In [39]:

```
# Calculate accuracy, precision, recall and F1 score of the model

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

orig.eval()

all_labels = []
all_predictions = []

with torch.no_grad():
    for i, (inputs, labels) in enumerate(test_loader):
        outputs = orig(inputs)
        _, predicted = torch.max(outputs, 1)

        all_labels.extend(labels.numpy())
        all_predictions.extend(predicted.numpy())

        if i >= 499:
            break

accuracy = accuracy_score(all_labels, all_predictions)
precision = precision_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)
recall = recall_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)
f1 = f1_score(
    all_labels, all_predictions, average="weighted", zero_division=0
)
```

```
print(f"Test images evaluated: {len(all_labels)}")
print(f"Test Accuracy: {accuracy:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall: {recall:.2%}")
print(f"F1 Score: {f1:.2%}")
```

```
Test images evaluated: 2000
Test Accuracy: 12.50%
Precision: 30.21%
Recall: 12.50%
F1 Score: 13.17%
```

Compare the two models. Which model performed better?

My first time running both of the models there was a less than 1% accuracy score on both of them. The error rate was around 5.4 steadily for all the all the batches. After some investigating I learned that the optimizer was not connected to the final layer which caused the weights to not be updated during training. After updating the optimizer to learn and adjust properly, the models loss began to decrease showing that the model was properly learning. With the corrected optimizer the models went from 5.1-5.4 loss range per batch to scores as low as 3.8.

The first model had an accuracy of 10.8% and a precision score of 24.6% while the second had an accuracy of 12.5% and precision score of 30.21%. Although these scores are not very high **The second model did a better job of accurately classifying images**